

NYCU Deep Learning Spring 2025

Lab7 Report

Policy-Based Reinforcement Learning

By 313554044 黃梓誠

授課教師：彭文孝、陳永昇、謝秉均

開課單位：資科工碩

永久課號：CSIC30153

Date Submission: May20, 2025

目錄

Introduction (5%).....	2
Your implementation (20%)	3
How do you obtain the stochastic policy gradient and the TD error for A2C?	3
How do you implement the clipped objective in PPO?	4
How do you obtain the estimator of GAE?.....	5
How do you collect samples from the environment?.....	6
How do you enforce exploration (despite that both A2C and PPO are on- policy RL methods)?.....	7
Explain how you use Weight & Bias to track model performance and the loss values (including actor loss, critic loss, and the entropy).....	8
Analysis and discussions (25%).....	10
Plot the training curves (evaluation score versus environment steps) for Task 1, Task 2, and Task 3 separately (10%).	10
Compare the sample efficiency and training stability of A2C and PPO.....	11
Perform an empirical study on the key parameters, such as clipping parameter and entropy coefficient (15%).	12
Additional analysis on other training strategies (Bonus up to 10%)	14

Introduction (5%)

在我這次的 Lab 7 實驗中實作兩種主流的基於策略的強化學習演算法：Advantage Actor-Critic (A2C) 和 Proximal Policy Optimization (PPO)。目的是在瞭解這些演算法的核心機制，並在 OpenAI Gym 的經典控制任務 Pendulum-v1 以及更具挑戰性的 MuJoCo Walker2d-v4 上評估性能。

在本報告中，我將首先詳細描述我對 A2C 和 PPO (包含 GAE 和 Clipped Objective) 的實作，包括 Network 架構、數學公式以及程式碼。接著，我會展示並分析在三個主要任務訓練結果，特別是 Training Curve、Sample Efficiency 和 Training Stability。其中，我會重點比較 A2C 和 PPO 在 Pendulum 環境上的表現差異。對於 Walker2d 任務，我還會探討 PPO 的關鍵超參數，以及使用不同 PPO 優化 trick 對學習性能的影響。

通過本次實驗，我驗證了 PPO 相較於 A2C 在 Training Stability 和 Sample Efficiency 上的優勢，並成功將 PPO 演算法應用於複雜的連續控制任務。

Your implementation (20%)

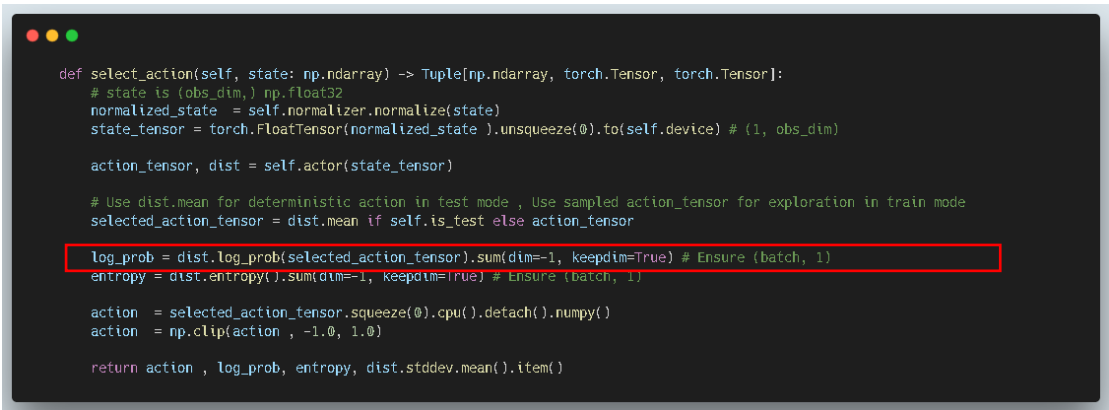
在本節中，我將詳細解釋我在 Task 1 (A2C on Pendulum), Task 2 (PPO on Pendulum), 和 Task 3 (PPO on Walker2d) 的實作。

How do you obtain the stochastic policy gradient and the TD error for A2C?

- Stochastic Policy Gradient:

在 A2C 實作中，Actor Network $\pi_{\theta}(a | s)$ 輸出一個高斯分佈的平均值 $\mu_{\theta}(s)$ 和對數標準差 $\log \sigma_{\theta}(s)$ 。在訓練的每一步，我從這個分佈 $N(\mu_{\theta}(st), \sigma_{\theta}(st))$ 中採樣一個動作 at 。

該動作的對數機率 $\log \pi_{\theta}(at | st)$ 在 `select_action` 方法中通過 `dist.log_prob(selected_action).sum(dim=-1)` 計算得到。



```
def select_action(self, state: np.ndarray) -> Tuple[np.ndarray, torch.Tensor, torch.Tensor]:
    # state is (obs_dim,) np.float32
    normalized_state = self.normalizer.normalize(state)
    state_tensor = torch.FloatTensor(normalized_state).unsqueeze(0).to(self.device) # (1, obs_dim)

    action_tensor, dist = self.actor(state_tensor)

    # Use dist.mean for deterministic action in test mode, Use sampled action_tensor for exploration in train mode
    selected_action_tensor = dist.mean if self.is_test else action_tensor

    log_prob = dist.log_prob(selected_action_tensor).sum(dim=-1, keepdim=True) # Ensure (batch, 1)
    entropy = dist.entropy().sum(dim=-1, keepdim=True) # Ensure (batch, 1)

    action = selected_action_tensor.squeeze(0).cpu().detach().numpy()
    action = np.clip(action, -1.0, 1.0)

    return action, log_prob, entropy, dist.stddev.mean().item()
```

而 Actor 的 Loss Function 定義為 $L_{actor}(\theta) = -(\log \pi_{\theta}(at | st) \cdot A^t + \beta \cdot H(\pi_{\theta}(\cdot | st)))$ ，其中 A^t 是 TD error (Advantage)， H 是 Policy entropy， β 是 entropy weight。Policy Gradient 就是該 Loss Function 對參數 θ 的梯度進行 Backpropagation (見下圖)

- TD error (Temporal Difference Error) / Advantage:

TD error δt (在 A2C 中直接作為 Advantage A^t) 的計算在 `update_model` 中完成。首先，Critic Network $V_w(s)$ 分別估算當前狀態 st 的價值 $V_w(st)$ 和下一個狀態 $st + 1$ 的價值 $V_w(st + 1)$ 。

接著，TD Target 計算為 $Q_t = r_t + \gamma * V * w(st + 1) * mask$ 這邊是用 mask 代表是否為中止狀態，終止狀態時 $Q_t = r_t$

最後，TD error (Advantage) 則為 $A^t = Q_t - Vw(st)$ 。

```
# TD Target: Q_val = r + gamma * V(s_next) * mask
current_value = self.critic(state_tensor) # V(s_t)
with torch.no_grad():
    next_value = self.critic(next_state_tensor) # V(s_{t+1})
    td_target = reward_tensor + self.gamma * next_value * mask_tensor

# Critic loss
# value_loss = F.smooth_l1_loss(current_value, td_target.detach())
value_loss = F.mse_loss(current_value, td_target.detach())

self.critic_optimizer.zero_grad()
value_loss.backward()
if self.max_norm > 0:
    torch.nn.utils.clip_grad_norm_(self.critic.parameters(), max_norm=self.max_norm)
self.critic_optimizer.step()

# Actor loss
advantage = (td_target - current_value).detach() # Advantage A_t = Q_val - V(s_t)
```

比較要注意的是在更新 Critic 時 td_target 需要 detach()，因為它是 Critic 的學習目標，這個目標不應該隨著 Critic 自身參數的更新而「移動」。而更新 Actor 時，advantage (由 td_target 和 current_value 計算得到) 需要被視為常數，所以構成它的 Critic 輸出項需要 detach()，以防止 Actor 的梯度回傳到 Critic。

How do you implement the clipped objective in PPO?

在 PPO 實作中，Clipped Surrogate Objective 的計算在 update_model 方法內的 ppo_iter 中完成。對於每個 mini-batch 中的樣本：

$$(st, at, A^t_{GAE}, \log \pi_{\theta_{old}}(at | st))$$

首先，使用當前的 Actor Network π_{θ} 計算新的對數機率 $\log \pi_{\theta}(at | st)$ 。再計算機率比率 ratio

$$rt(\theta) = \exp(\log \pi_{\theta}(at | st) - \log \pi_{\theta_{old}}(at | st))。$$

```

# calculate ratios
_, dist = self.actor(state)
new_log_prob = dist.log_prob(action)
ratio = (new_log_prob - old_log_prob).exp()

```

接著計算兩個替代目標：

$$L1 = rt(\theta)A^tGAE \quad \text{和}$$

$$L2 = clip(rt(\theta), 1 - \epsilon, 1 + \epsilon)A^tGAE。$$

最後，PPO 的 Actor Loss 計算時需要取負號，因為我們要最小化 Loss，因此

$$L_{actorCLIP} = -Et[\min(L1, L2)] - \beta H(\pi\theta(\cdot|st))。$$

```

# actor_loss
# adv (advantages) shape: [mini_batch_size, 1]
surr1 = ratio * adv
surr2 = torch.clamp(ratio, 1.0 - self.epsilon, 1.0 + self.epsilon) * adv

# PPO maximizes the objective, so for minimization loss, we take the negative.
actor_loss_clipped = -torch.min(surr1, surr2).mean()

# Entropy bonus
entropy_bonus = dist.entropy().mean() # .mean() to get a scalar average entropy for the batch
actor_loss = actor_loss_clipped - self.entropy_weight * entropy_bonus

```

How do you obtain the estimator of GAE?

Generalized Advantage Estimator (GAE) 的計算封裝在 `compute_gae` 函數中。

該函數接收一個 rollout 期間收集到的 `rewards`、`masks (1-done)`、`values` 列表，以及 rollout 結束時的下一個狀態的價值估計 `next_value (Vs(T + 1))`。

GAE 的計算從一個 rollout 序列的最後一步 `T` 開始，反向反覆運算到第一步 `t=0`。在每個 Time step `i`，首先計算 TD error

$$\delta i : \delta i = r_i + \gamma V_s(i + 1) \cdot \text{mask}_i - V_s(i)$$

然後，GAE 值 A^iGAE 通過以下遞迴方式計算：

$$A^iGAE = \delta i + \gamma \lambda \cdot \text{mask}_i \cdot A^{i+1}GAE$$

其中， $A^T + 1GAE$ 被初始化為 0。

最終，我們計算的目標回報 Returns 用於訓練 Critic，其定義為

$$R_i = A^iGAE + V_s(i)。$$

Advantage A^iGAE 則在 `update_model` 中通過 `returns - values_old` 得到。

```
def compute_gae(
    next_value: list, rewards: list, masks: list, values: list, gamma: float, tau: float) -> List:
    """Compute gae."""
    # rewards, masks, values 是 list of tensors

    gae_values = [torch.zeros_like(values[0]) for _ in range(len(values) + 1)] # 存儲 GAE estimates
    gae_returns = [torch.zeros_like(values[0]) for _ in range(len(values))] # 存儲 target returns R_t
    gae = torch.zeros_like(values[0]) # 初始化 gae 累加器

    # 從最後一步 (N-1) 往前計算
    for i in reversed(range(len(rewards))): # i from rollout_len-1 down to 0
        if i == len(rewards) - 1:
            v_next_s = next_value # V(s_{rollout_len})
        else:
            v_next_s = values[i+1] # V(s_{i+1})

        # TD error: delta_i = r_i + gamma * V(s_{i+1}) * mask_i - V(s_i)
        delta = rewards[i] + gamma * v_next_s * masks[i] - values[i]

        # GAE: A_i = delta_i + gamma * tau * mask_i * A_{i+1}
        gae = delta + gamma * tau * masks[i] * gae
        current_return = gae + values[i]
        gae_returns[i] = current_return # 存儲 R_i

    return gae_returns
```

How do you collect samples from the environment?

- (1) 對於 A2C (Task 1)，我在 `train` 迴圈的每個 episode 中，逐個 step 與環境交互。在 `select_action` 中選擇動作 at 並記錄 $\log\pi(at|st)$ 和 st ，然後在 `step` 方法中執行 at 得到 rt , $st + 1$, $done$ ，並每一步都調用 `update_model`。

```
while not done:
    self.total_step += 1
    action, log_prob_tensor, entropy_tensor, current_std_mean = self.select_action(state)
    next_state, reward, done = self.step(action)
```

- (2) 對於 PPO (Task 2 和 Task 3)，我在 `train` 迴圈中，首先有一個外層迴圈控制 PPO 的總更新次數 (或者總訓練步數)。在每個 PPO 更新週期內，有一個內層迴圈負責收集 `self.rollout_len` 長度的經驗數據。他會每個 step 調用 `select_action` 根據當前策略 π_θ 選擇動作 at ，並同時獲取 Critic 對當前狀態

st 的價值估計 $V(st)$ 以及動作的對數機率 $\log \pi \theta(at | st)$ 。當收集滿 `rollout_len` 步的數據後，調用 `update_model` 進行學習。

```
for _ in range(self.rollout_len):
    self.total_step += 1
    action = self.select_action(state)
    next_state, reward, done = self.step(action)

    state = next_state
    score += reward

    if done[0][0]:
        wandb.log({
            "step": self.total_step,
            "train_score_vs_env_steps": score
        })
        episode_count += 1
        state, _ = self.env.reset() # 不限制 seed
        state = np.expand_dims(state, axis=0)
        scores.append(score)
        score = 0

actor_loss, critic_loss, entropy_bonus = self.update_model(next_state)
```

How do you enforce exploration (despite that both A2C and PPO are on-policy RL methods)?

由於 A2C 和 PPO 都是 on-policy 方法，它們的探索主要來自於策略本身的隨機性，因此 exploration 主要來自以下原因：

1. 高斯策略的採樣：Actor Network 輸出的是高斯分佈的均值 μ 和標準差 σ (或 $\log \sigma$)。在訓練階段 (`is_test=False`)，我通過 `dist.sample()` 從這個高斯分佈中採樣動作，而不是直接取均值。這個採樣過程自然地引入探索。標準差 σ 的大小直接決定了探索的幅度。
2. Entropy Regularization：為了進一步鼓勵探索並防止策略過早收斂到確定性，我在 Actor 的 Loss Function 中加入了一個熵獎勵項：

$$-\text{entropy_weight} * H(\pi \theta(\cdot | st))。$$

其中 H 是策略的熵，`entropy_weight` 是一個用於控制熵獎勵強度的超參數。較大的熵意味著策略更隨機，探索性更強。熵的計算是通過 `dist.entropy().mean()` 得到的。

```
def forward(self, state: torch.Tensor) -> Tuple[torch.Tensor, torch.distributions.Normal]:
    x = self.hidden_layers(state)

    mean = self.mean_layer(x)

    log_std = self.log_std_layer(x)
    log_std = torch.clamp(log_std, -2, 1.5)
    std = torch.exp(log_std) + 1e-8

    dist = Normal(mean, std)
    action = dist.sample() # PPO 通常在收集數據時也從分佈中採樣

    return action, dist
```

log_std 的學習：在我的 Actor Network 中，log_std 是可以學習的（無論是 state-dependent 還是 state-independent）。通過梯度更新，Network 可以自行調整策略的隨機性。我還對 log_std 進行了 clamp 操作，以維持其在一個合理的範圍內，避免標準差過小（探索不足）或過大（過度隨機）。

```
class Actor(nn.Module):
    def __init__(self, in_dim: int, out_dim: int, hidden_dims: List[int] = [128, 64], log_std_min: int = -20,
log_std_max: int = 2, dropout_p: float = 0.0):
        super(Actor, self).__init__()
        self.dropout_p = dropout_p

        layers = []
        prev_dim = in_dim
        for hidden_dim in hidden_dims:
            layers.append(nn.Linear(prev_dim, hidden_dim))
            layers.append(nn.ReLU())
            if self.dropout_p > 0:
                layers.append(nn.Dropout(p=self.dropout_p))
            init_layer_uniform(layers[-3] if self.dropout_p > 0 else -2)
            prev_dim = hidden_dim

        self.hidden_layers = nn.Sequential(*layers)

        self.mean_layer = nn.Linear(prev_dim, out_dim)
        self.log_std_layer = nn.Linear(prev_dim, out_dim)

        init_layer_uniform(self.mean_layer, init_w=1e-3)
        init_layer_uniform(self.log_std_layer, init_w=1e-3)
```

Explain how you use Weight & Bias to track model performance and the loss values (including actor loss, critic loss, and the entropy).

在實驗中我全面使用 Weights & Biases 來追蹤和可視化訓練過程。因此可以很好知道不同策略以及超參數會如何影響結果，這對我瞭解 A2C 和 PPO 這兩個演算法的過程中很有幫助，也優化了工作流程。

1. 在 update_model (對於 A2C 或 ppo_iter 迴圈)內部，我用 wandb.log() 記錄了：

- i. "losses/actor_loss" : Actor 的平均 Loss
- ii. "losses/critic_loss" : Critic 的平均 Loss
- iii. "charts/entropy_bonus" : 策略的平均熵
- iv. "charts/policy_std_mean" : 策略的平均標準差。
- v. "charts/learning_rate_actor" 和 "charts/learning_rate_critic"

如果使用了 Linear learning rate annealing，同時也會記錄當前 Step 中 actor 和 critic 的 learning rate。這些指標都以 total_env_steps 作為 x 軸。

另外，在每個 episode 結束後，我會記錄：

- vi. "rollout/episodic_return, length" : 代表 rollout 過程中的完整 episode 的回報和長度。
- vii. "eval/average_score" : Eval 階段的平均分數

因為我觀察到 training 時期的 reward 通常較為震盪(畢竟每次 env 完全隨機)。因此我的策略是每 5 個 eps 後，會調用專用的評估函數 test_agent_performance，定期評估得到的在不同 seed 測試下的平均分數。這些指標也以 total_env_steps 作為 x 軸，這樣可以清晰地看到模型性能隨訓練過程的變化，可以讓我知道何時應該保存模型。

2. Sweep 優化繁瑣的參數工程：對於超參數調整，我使用了 wandb.sweep 功能。我為三個任務定義了 YAML 設定檔，指定了要搜索的參數空間（例如 Learning Rate、Network Structure architecture_key、entropy_weight、optimizer_choice 等）和優化目標（例如最大化 eval/average_score）。wandb agent 會自動運行多個實驗並記錄結果，方便我比較和選擇最佳配置。

Analysis and discussions (25%)

Plot the training curves (evaluation score versus environment steps) for Task 1, Task 2, and Task 3 separately (10%).

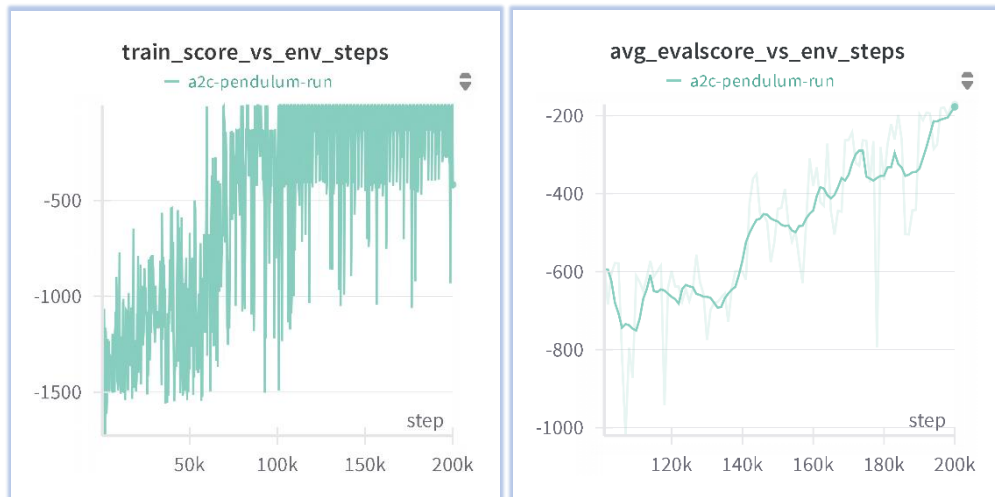


Fig. Task1 training curves & eval curves versus env steps

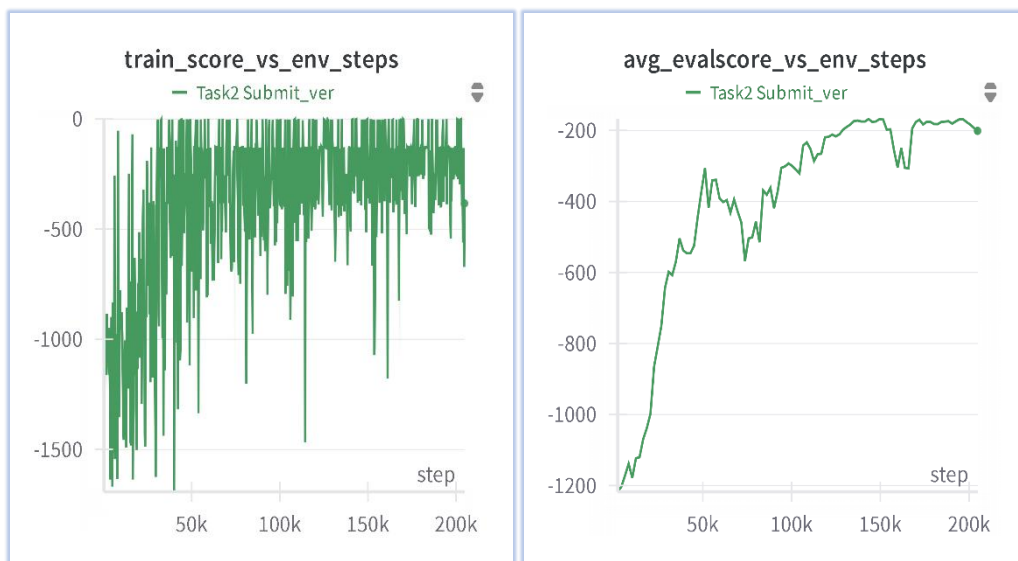


Fig. Task2 training curves & eval curves versus env steps

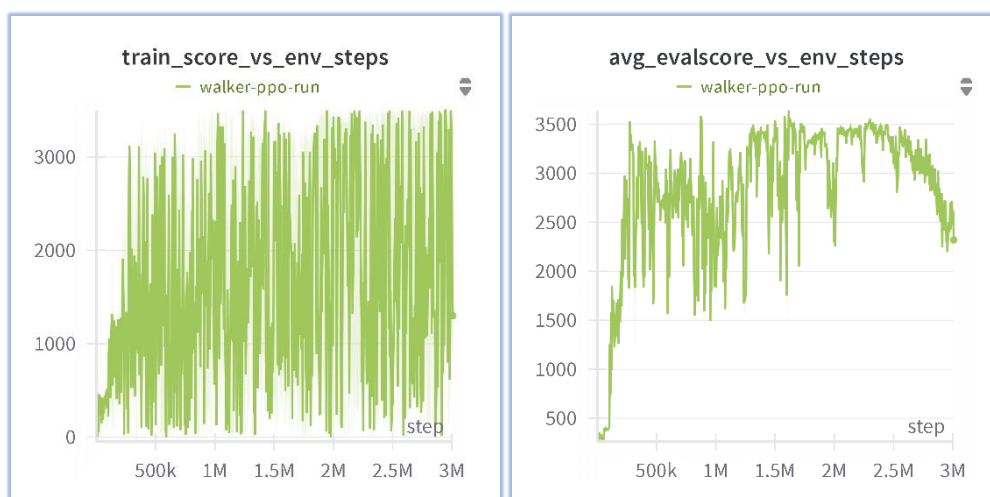


Fig. Task3 training curves & eval curves versus env steps

Compare the sample efficiency and training stability of A2C and PPO.

- Sample Efficiency(A2C vs PPO) :

從 Fig. Task1 和 Fig. Task2 的比較可以看出，PPO 在約 50k steps 時達到了 -150 的平均回報，而 A2C 則需要約 100k steps，顯示了 PPO 在此任務上具有更高的 Sample Efficiency。

- Training Stability(A2C vs PPO) :

觀察 Loss Curve (如下圖)，PPO 的訓練過程明顯比 A2C 更為穩定，A2C 的 Actor 和 Critic Loss 容易出現較大的尖峰或震盪，而 PPO 的 Loss 曲線更為平緩。



Fig. Loss Comparison between Task1and Task2

- Sample Efficiency & Training Stability On task3 (A2C vs PPO) :

可以看到採用 1-step return 的 A2C，無法在 Walker2d-v4 中學會任何動作，即使一開始偶有震盪，前期就因為 loss 爆掉導致訓練失敗，相較之下，採用 PPO-

Clip 則在 Walker2d-v4 任務中則在訓練初期的數百 K steps，便呈現持續且穩健的增長趨勢，最終在三百萬步的訓練後，達到了超過三千分的高水準性能。其學習曲線雖然也存在強化學習固有的波動，但**整體上升趨勢明確**，顯示其 Sample Efficiency、Training Stability 以及策略的持續優化能力



Fig.Task3 Comparison between A2C and PPO

Perform an empirical study on the key parameters, such as clipping parameter and entropy coefficient (15%).

- Clipping Parameter (ϵ)：在 PPO pendulum 中，調參是一個困難的任務，我使用 sweep 嘗試不同的 ϵ 值 (0.05, 0.1, 0.2, 0.3)。我發現 $\epsilon = 0.1$ 在性能和穩定性之間取得了較好的平衡，始終在領先地位(紅線)。過小的 ϵ 可能導致學習過慢(綠線)，過大的 ϵ (橘線)雖然前期像紅線般探索較快但也較不穩，失去 PPO 穩定的優勢，使其接近於傳統的 Policy Gradient。

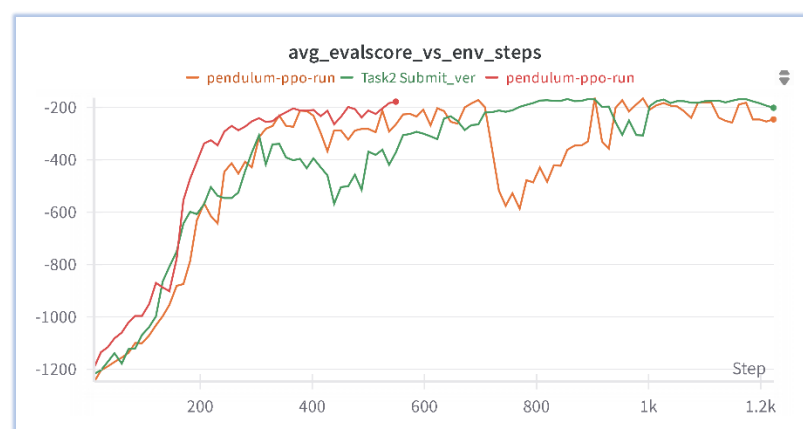


Fig. Eval Curve 和最終性能的影響

- Entropy Coefficient (entropy_weight)：對於 Walker2d，我預期它適合非常小的熵權重 ($5e-6$ ，甚至為 0) 可能會比較好，因為任務需要精確的控制。因此這裡主要討論 PPO pendulum 任務，我使用不同 entropy_weight (例如 $1e-5$, $5e-6$, 0.01) 可以看到發現即使 entropy 很大 ex:0.01，也對訓練過程沒有影響，可見 PPO 對於 entropy 在該任務上是有足夠穩定度的

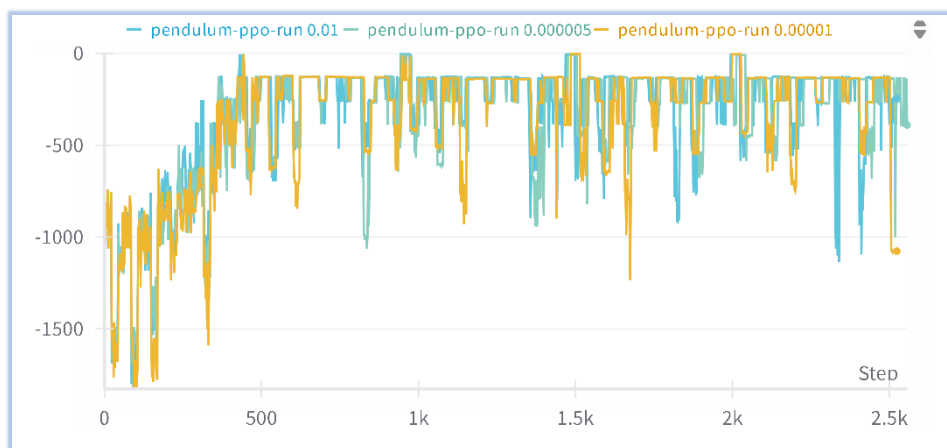


Fig. Training curve versus different Entropy weight

- Network Structure：在 Walker2d 中，我都是使用兩層 linear+RELU，命名方式為 NN 大小，我發現 NN 架構對性能的影響非常明顯，原先我預期較小的 NN 架構可能較有機會在 1M 達成 2500 分，因為它不需要學習那麼多參數，但結果發現過小的 NN(64-32)會無法學會精細的控制任務，而過大的 NN 則和較小的 NN，是可以達到 2500 分的，他們有著差不多的訓練曲線，並且最後我找到最佳的層數為 128-128，可以看到他在 250K 就逐漸突破 3k 分數，可見他是適合該任務的大小

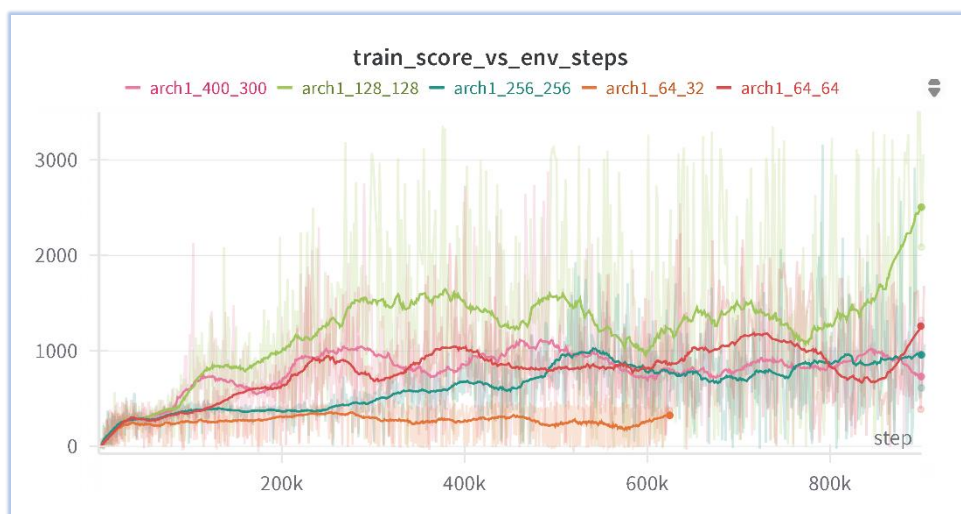


Fig. Training curve versus NN architecture

但是如果不限時間，可以看到較大的 NN(400-300)最終性能會超越 128-128，也證明前期快速收斂不一定好，應該拉長時間尺度以更好的發現不同架構對**最終性能**的影響

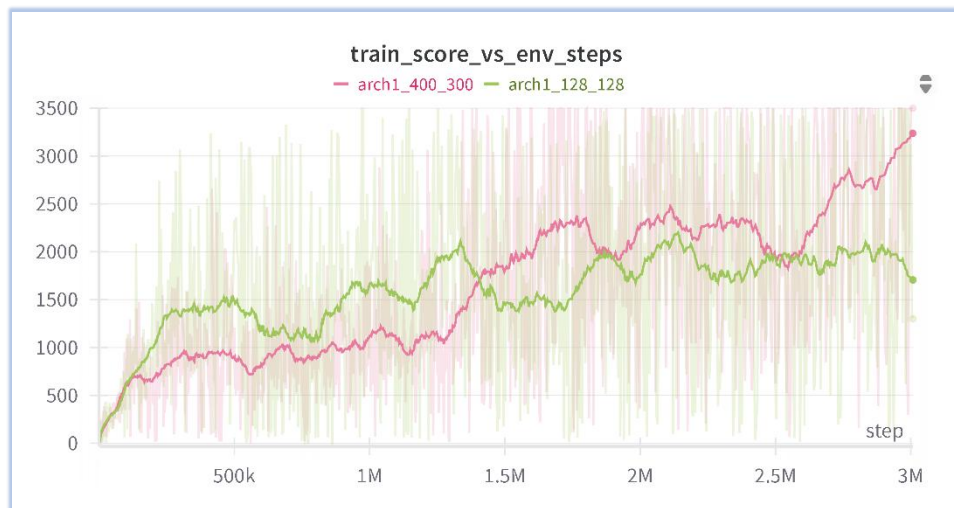


Fig. Training curve versus NN architecture

Additional analysis on other training strategies (Bonus up to 10%)

- 不同的 Network 啟動函數 (ReLU vs Tanh) 對性能的影響(見下圖)

在 Task3 Walker2d-v4 中，我的 Actor 和 Critic NN 都是兩層 Linear+ activation，我原先以為 Tanh 能夠把 obs 和 action 縮放到某個區間，會適合該任務，但是實際操作後發現相差甚遠，以下是我的分析: (圖片中 Tanh-Relu 表示 Actor 的 activation 是 Tanh, 而 Critic 為 Relu)

雖然 Tanh 輸出範圍在 $[-1, 1]$ 之間，但在深度網路和特定 RL 任務中，它可能不如 ReLU 的原因，我覺得是梯度消失 (Vanishing Gradients)。Tanh 在其輸入值較大或較小的區域，其導數（梯度）會變得非常小。並且如果多層串聯(在我的設計中為兩層)，梯度也衰減的更加迅速，最後導致梯度消失。

對於 Actor 而言，梯度消失代表策略難以根據獎勵信號進行有效調整和優化。同樣地，對於 Critic 而言，如果 Critic 網路的隱藏層使用 Tanh 並遭遇梯度消失，它就很難準確地學習狀態價值函數 $V(s)$ ，從而**無法提供準確的 Advantage 估計給 Actor**，進而加劇影響 Actor 的學習。圖中紫色線 (Relu-Tanh) 和粉紅色線 (Tanh-Tanh) 的糟糕表現，即解釋了 Critic 使用 Tanh 時可能遭遇了這個問題。

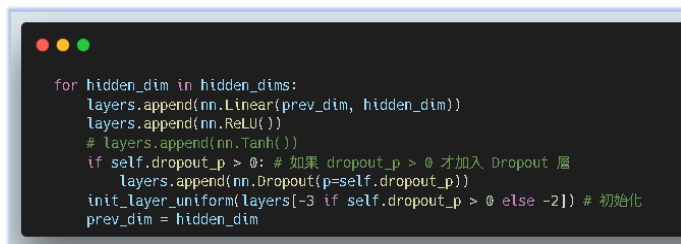


Fig. NN Design

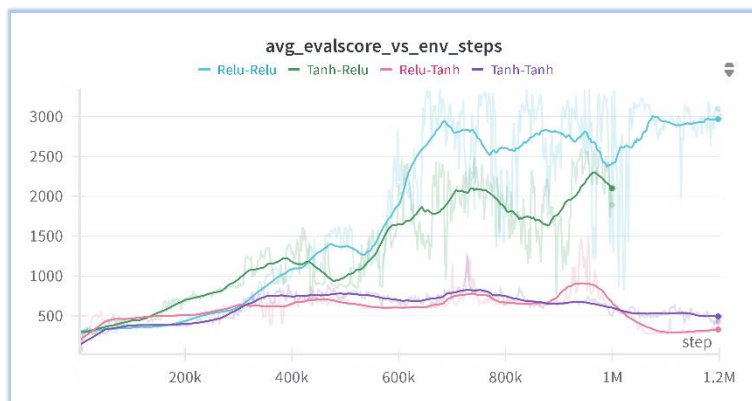


Fig. NN activation function & performance

• Obs Clipping & Advantage Normalization 技巧

下圖右方顯示 Advantage Normalization 和 Obs Clipping 技巧的引入前後對 training 過程的影響。原始（淺藍色線）的學習軌跡較為不確定，儘管偶爾能觸及較高的性能水準，但隨即而來的劇烈下滑揭示了其策略的脆弱性和更新梯度的不穩定性，整體表現更像是在隨機的成功與失敗間擺盪。

相較之下，採用了 Advantage Normalization 的 Agent（紅色線）不僅以更快的速度達到-150 分以上，並克服原始設定中的性能抖動。其學習曲線顯得平滑且持續向上，即便在高分區間也維持著相對的穩定性。

這意味著 Obs Clipping & Advantage Normalization 的改進能有效平抑原始策略梯度中可能存在的過大或過小的噪聲，使得學習方向更加明確，最終塑造出一個更為穩健的策略。

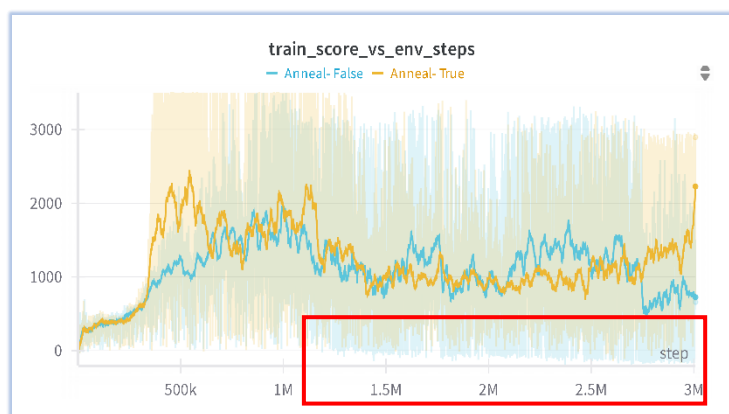


Fig. Learning Rate Annealing comparison

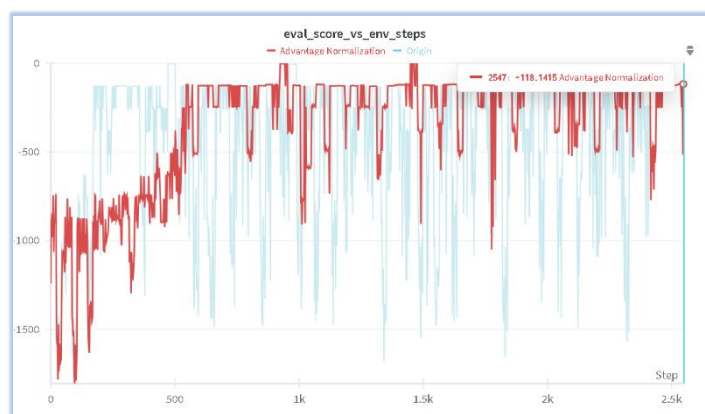


Fig. Advantage Normalization

- **Learning Rate Annealing 策略對收斂速度和最終性能的正面影響(上圖左)**

該實驗探討 Learning Rate Annealing 策略對強化學習 Agent 在特定連續控制任務中收斂速度及最終性能的影響。其中橘色線代表啟用 Learning Rate Annealing (Anneal-True)，而淺藍色線則代表使用固定 Learning Rate (Anneal-False)。

在訓練過程約 500k steps 之後，固定 Learning Rate 的 Agent (淺藍色線) 雖然一度達到約 1500 至 1800 分的性能水準，但其後續過程充滿了劇烈的性能震盪，**同時也容易有超低分**，甚至在訓練後期出現了明顯的性能衰退和不穩定現象，至三百萬步時分數已回落至較低區間。

相較之下，實施了 Learning Rate Annealing 的 Agent (橘色線) 則表現出更強的學習能力和穩定性。在訓練的後半段，雖然同樣存在強化學習固有的性能波動，**但其最低值始終顯著優於固定 Learning Rate 組**。更為重要的是，在接近 3M steps 的訓練尾聲，採用 Annealing 策略的 Agent 再次攀升，顯示出其策略仍在持續優化。

綜上所述，Learning Rate Annealing 在該任務中，透過在訓練過程中逐步減小 Learning Rate，有助於 Agent 在探索的初期進行較大幅度的更新，而在學習後期則能更精細地調整策略參數，從而避免了因固定 Learning Rate 可能導致的在最優點附近震盪或過早收斂至次優策略的問題，最終促成了更穩定的學習成果。